

Technical Assessment: Stock Opname App

1. Overview

Store staff need a reliable web application and API to perform periodic physical stock counts (Stock Opname) and record actual inventory count across multiple store locations.

Your task is to build a backend API service (preferably Express.js + PostgreSQL) and a supporting frontend application (e.g., Next.js, React) that manages the complete Stock Opname lifecycle, from snapshotting existing stock levels to executing inventory changes upon manager approval.

2. Main Requirements

2.1. Master Data & Authentication

- Manage products with SKU identifiers and current stock quantities.
- Implement authentication with at least two roles: Store Staff and Store Manager.
- Staff belongs to a store, and performs stock opname sessions on their store. Managers can initiate stock opname sessions, review submitted counts, and approve or reject them.
- Implement authentication and session/token handling with access controls based on the above.

2.2. Stock Opname Business Logic

The lifecycle has three stages: **initiation**, **count submission**, and **approval**. How you implement them is your decision:

1. **Initiation:** A manager starts a stock opname session. Some things you might consider:
 - When a manager initiates a session, what should happen?
 - Regarding the stock, how should the stock be maintained? Should it be per store or in total?
 - Is initiation even necessary or should a stock opname just use the current date? What are the pros/cons to this approach?
2. **Count Submission:** Staff submit counts for some or all items in the stock opname session in a single batch. Some things you might consider:
 - Does this submission change stock right away?
 - What if a submission includes multiple of the same item?
 - Is it necessary for staff to be able to edit their submission?
 - Is it possible or necessary or allowed for staff to submit multiple submissions?
 - Are there concerns if there are a large number of items in stock opname session?
 - Other edge cases.
3. **Approval:** After staff submits a stock opname session for their store, the manager then

reviews then approves/rejects the submission. Some things you might consider:

- What happens when the manager approves a submission?
- What if the process after approval takes a long time to finish?
- What if the process after approval fails?
- What happens if the manager approves multiple submissions from the same store with different values?
- What should the manager do if they review and don't agree with the submission?
- Other edge cases.

The explanation above is intentionally vague and leaves room for your interpretation. We expect you to think of the problem and choose and develop the solution. You are also encouraged to consider and handle edge cases, or add some features you think are useful (for example, bulk upload stock opname so users don't need to input in form manually), etc.

2.3. Frontend

A responsive web interface demonstrating:

- Login and different functionality based on the role.
- A staff view for entering all counts for a stock opname session for their store.
- A manager view for reviewing sessions and approving/rejecting.

Feel free to use the tools and libraries you usually use or what's commonly used for frontend such as Tailwind, Shadcn, or others.

3. AI Policy & Transparency

You are allowed to use AI tools (e.g., Copilot, ChatGPT, Claude) as coding assistants, but **you are expected to drive the logic and implementation. Note that we are hiring the software developer, not the AI.**

The brief leaves several things unspecified on purpose. What you do with those gaps is the core part of what we're assessing. This is why the NOTES.md deliverable is required.

A submission that we deem is what an AI would produce with no visible sign of your review and guidance, will be automatically rejected.

4. Deliverables

1. **Timeframe:** Maximum 1 week.
2. Send a link to a public github repository containing:
 - Backend and Frontend source code. May be in 1 or separate repos.
 - A README.md with instructions on the tech stacks used, how to run the application.
 - Implementation notes doc in NOTES.md. **Please answer the followings with your own words, not AI-generated:**
 - The reasoning of your interpretation and choices you made in section 2
 - Walk through what happens from the moment a manager clicks approve

to the moment stock levels are updated. What could go wrong at each step, and what did you consider and do about it?

- What happens to items that are left uncounted by the staff during a stock opname session? For example, if they forget or if the items don't exist in their store. Do you allow submission with empty value for items? Do you treat them as zero stock, ignore them (keep previous stock), or handle them another way? And what is your reasoning?
- What are some edge cases you thought of and how did you handle them?
- What is one thing you added to this project that you think is important? Why?